

SYSTEM CARD

AI Support Ticket Analyst

A natural-language query and anomaly-detection system over customer support ticket data, where every figure a user sees is computed by SQL or statistics — never by the language model.

Version 1.0.0 — Stable

LLM: Groq · openai/gpt-oss-120b

FastAPI + Streamlit

381 offline tests

100% benchmark pass rate

Single-user, local deployment

RELEASE DATE

18 September 2026

CONTEXT

DOTMappers AI Engineer
Assessment

AUTHOR

Vijaya Karthik

Contents

1	System Summary	3
2	Intended Use and Users	4
3	System Architecture	5
4	The Language Model Component	7
5	Data	8
6	Safety, Security and Guardrails	10
7	API Surface and Configuration	12
8	Evaluation and Performance	14
9	Known Limitations and Risks	16
10	Versioning and Change History	18
11	Dependencies	19
12	Ownership and Maintenance	20

Page numbers are approximate and reflect this document's own pagination at time of publishing.

1. System Summary

The AI Support Ticket Analyst ingests a 500-row customer support ticket dataset and answers natural-language questions about it, while separately flagging operational anomalies (unusually slow resolutions and breached service-level agreements). It exposes both capabilities through a REST API and a web UI, and is designed to run at zero monetary cost using a free-tier hosted language model.

SYSTEM NAME AI Support Ticket Analyst	VERSION 1.0.0 (Semantic Versioning)
INTERFACES REST API (FastAPI) + Web UI (Streamlit)	LANGUAGE MODEL Groq free tier — openai/gpt-oss-120b
DATABASE SQLite, read-only at query time	STARTUP <code>python run.py</code> (single command)
DEPLOYMENT SCOPE Single-user, local machine only	COST \$0 — free tiers only, no paid services

The core design principle

The language model understands the question. It never calculates the answer. The model is used only for two tasks: translating a natural-language question into a SQL query (or a choice of anomaly detector), and phrasing a computed result into a sentence. Every number shown to a user is computed by SQLite or by pandas, and a dedicated verification step checks — on every single response — that every figure in the model's written answer can be traced back to that computed evidence.

What the system does

Capability	How it is implemented
Natural-language question answering	LLM translates the question into a validated, read-only SQL query; SQLite computes the result; the LLM phrases it into prose, which is then verified against the evidence.
Anomaly detection	Two deterministic statistical detectors (resolution-time outliers via Tukey's IQR fence; SLA breaches via a fixed business rule) implemented in pandas, with no language model involved at all.
REST API	Four FastAPI endpoints — <code>/health</code> , <code>/schema</code> , <code>/anomalies</code> , <code>/query</code> — with interactive OpenAPI documentation at <code>/docs</code> .
Web UI	A Streamlit application that is a pure HTTP client of the API — it contains no independent business logic, so the API and UI cannot disagree with one another.

2. Intended Use and Users

Primary intended use

- Answering ad-hoc, natural-language analytical questions about a fixed, historical customer-support ticket dataset (e.g. counts, averages, rankings, comparisons across time periods).
- Surfacing operational anomalies — slow resolutions and SLA breaches — for review by a support operations analyst or manager.
- Serving as a demonstration / evaluation artifact showing a defensible architecture for grounding an LLM's output in real, verifiable data.

Intended users

- A single local operator running the system on their own machine (support analyst, engineering evaluator, or developer).
- An evaluator (e.g. a technical interviewer) reviewing the system's design, correctness and reasoning as a work sample.

Out of scope

- **Not a production, multi-tenant service.** There is no authentication, no per-user rate limiting, and both interfaces bind to `localhost` only.
- **Not for datasets other than the shipped one.** The SQL guard, prompts, and anomaly detectors are all written against this dataset's fixed schema; there is no file-upload path.
- **Not a source of real-time operational truth.** The dataset is a static historical snapshot ending 2024-03-30; "now" is simulated via a pinned time anchor (`AS_OF`), not the real clock.
- **Not intended to answer questions outside the ticket dataset.** General-knowledge questions and requests to modify data are refused by design.

3. System Architecture

Component overview

Component	Responsibility	File
Settings	Typed configuration loaded once from <code>.env</code> ; invalid values fail at startup, not mid-request.	<code>app/config.py</code>
Ingestion	CSV → SQLite, explicit NULL handling, the <code>AS_OF</code> time anchor, read-only connections.	<code>app/data.py</code>
SQL Guard	Validates model-generated SQL before execution (barrier 1 of 2).	<code>app/sql_guard.py</code>
Anomaly Detectors	Deterministic statistical detectors behind an extensible registry. No LLM involved.	<code>app/anomalies.py</code>
Prompts	System prompt, tool schemas, and narration prompt sent to the model.	<code>app/prompts.py</code>
Orchestration	Tool-calling pipeline, retry policy, bounded recovery paths.	<code>app/llm.py</code>
Grounding	Verifies every number in a model-written answer against the evidence it was shown.	<code>app/grounding.py</code>
Schemas	Pydantic request/response models; generates the OpenAPI docs.	<code>app/models.py</code>
API	FastAPI app: composition, lifecycle, logging, error-to-HTTP mapping.	<code>app/main.py</code>
UI	Streamlit interface; pure HTTP client with no business logic.	<code>ui/streamlit_app.py</code>
UI Formatting	Display logic — escaping model text, dates, badges — kept separately so it is unit-testable.	<code>ui/formatting.py</code>
Launcher	Single-command startup for API + UI together; watches both processes.	<code>run.py</code>

How a question is answered

The pipeline is deliberately **bounded**: two model calls on the happy path, four at most. Every recovery path is capped at exactly one additional attempt, giving a predictable token ceiling regardless of how a question goes wrong.

- 1 **Choose an action** — the model is forced (`tool_choice="required"`) to call one of two tools: `query_tickets` or `detect_anomalies` . It cannot answer directly in prose. One retry with an explicit instruction if it replies in prose instead; a genuine refusal is honoured and shown as a fixed message.
- 2 **Validate** — generated SQL passes through `sql_guard` : single statement, SELECT-only, no wall-clock dates, no overflowing month arithmetic. One repair attempt if rejected, guided by the specific validation error.
- 3 **Execute** — SQLite, opened read-only (`mode=ro`), row-capped at 500, time-limited to 5 seconds per statement.
- 4 **Narrate** — the model phrases the real rows (or anomaly report) into a natural-language answer.
- 5 **Verify (grounding check)** — every number in the answer must appear in the evidence the model was shown. If any figure is unexplained, the narration is discarded and replaced with a deterministic, factually correct summary.

Why a bounded pipeline, not an open-ended agent

Groq's free tier enforces an 8,000-token-per-minute budget. An open-ended agent loop could let one confused question consume the entire budget for every subsequent question. Bounding every recovery path at one extra attempt gives a predictable ceiling: approximately 1,500 tokens for a normal question and roughly 3,000 tokens in the worst case.

4. The Language Model Component

Property	Value
Provider	Groq (hosted inference, free tier)
Model	<code>openai/gpt-oss-120b</code> — the largest chat model available on Groq's free tier at build time; Llama models were confirmed to no longer be offered free.
SDK	<code>groq</code> Python SDK v1.7.0, OpenAI-compatible native tool calling
Temperature	0.0 (deterministic sampling — a data question has one correct answer)
Reasoning effort	<code>low</code> — measured at 12 reasoning tokens vs. 35 at the default "medium," with identical SQL output on this schema; prompt size, not reasoning depth, dominates the token budget
Tool-call token ceiling	700 tokens
Narration token ceiling	400 tokens
Tool selection	<code>tool_choice="required"</code> — the model cannot reply in free-form prose when a tool call is expected
Fallback model options	<code>qwen/qwen3.8-27b</code> , <code>openai/gpt-oss-20b</code> (documented in <code>.env.example</code> ; not enabled by default)

Role in the system

The model performs exactly two functions and no others: **(a)** translating a natural-language question into either a SQL query or a choice of anomaly detector with arguments, and **(b)** phrasing a real, already-computed result into a natural-language sentence. It never performs arithmetic, aggregation, or comparison itself — those operations happen in SQLite or pandas, outside the model entirely.

Known behavioural characteristics of the model in this system

- **Non-deterministic despite temperature 0.** The same question has been observed to be phrased or routed differently between runs of the live model.
- **Provider-side refusal handling.** Groq rejects the whole HTTP request (not a normal message) when `tool_choice="required"` is set and the model chooses to reply in prose instead, placing its intended reply in a `failed_generation` error field. This project's client code specifically detects and handles that provider behaviour rather than treating it as a transport failure.
- **Reasoning-model token consumption.** As a reasoning model, `gpt-oss-120b` spends output tokens "thinking" before producing visible content; an insufficient `max_tokens` budget can silently return empty content rather than an error.
- **The model is never trusted unverified.** Every written answer is passed through the grounding check (Section 6) before being returned to the user.

5. Data

Dataset

SOURCE FILE

data/support_tickets.csv

ROW COUNT

500 support tickets

DATE RANGE

Ends 2024-03-30 18:06 (fixed historical snapshot)

STORAGE

SQLite table `tickets`, rebuilt from the CSV at every startup

Schema

Column	Type	Nullable	Description
ticket_id	TEXT (primary key)	No	Unique ticket identifier
created_at	TEXT (ISO, sortable)	No	When the ticket was raised
category	TEXT (enum)	No	Billing / Technical / General
priority	TEXT (enum)	No	Low / Medium / High / Critical
status	TEXT (enum)	No	Open / Resolved / Escalated
response_time_hrs	REAL	No	Hours until first agent response
resolution_time_hrs	REAL	Yes	Hours until resolution — NULL while unresolved (Open or Escalated)
agent_id	TEXT	No	Assigned support agent
customer_rating	INTEGER (1–5)	Yes	Satisfaction rating — NULL while unresolved
issue_summary	TEXT	No	Short description of the reported issue

The AS_OF time anchor

The dataset ends on 2024-03-30, but natural-language questions use relative expressions such as "this week" or "this month." Resolved against the real wall-clock date, every such question would return zero rows — a functioning system that appears broken during a live demonstration. `AS_OF` is a configurable reference "now," defaulting to the dataset's own latest `created_at` timestamp, against which all relative-date expressions are resolved instead of the system clock.

Data integrity guarantees enforced at ingestion

- A blank `resolution_time_hrs` or `customer_rating` becomes real SQL `NULL` — never `0.0` and never `NaN` — because either would silently corrupt every average computed over the column.
- Durations are rejected if negative, non-finite, or the literal strings `"nan"` / `"inf"` (which Python's `float()` otherwise accepts silently).

- Categorical fields (`category` , `priority` , `status`) are validated against a fixed enum at load time; an unrecognised value fails startup rather than surfacing as a silently wrong query result later.
- Ticket ids are validated as unique; duplicates fail ingestion with the offending id and line number named explicitly.
- Ingestion uses Python's standard-library `csv` module rather than pandas, specifically because pandas' automatic dtype inference is a known source of the NaN-vs-NULL defect this system is designed to avoid.

Known data caveat. The assessment brief's own schema description implies an `Escalated` ticket carries a resolution time and rating like any other row. In the actual shipped CSV, every `Escalated` row has both fields blank. The system follows the real data — treating `Escalated` as unresolved — rather than the brief's illustrative description.

6. Safety, Security and Guardrails

Two independent barriers against unsafe SQL

Because the language model writes SQL, that SQL is treated as untrusted input — not on the assumption the model is adversarial, but because a model can be wrong, and a user can ask it to do something destructive. Two barriers exist, deliberately sharing no code, so a defect in one cannot defeat the other.

Barrier	Mechanism	Purpose
1. SQL Guard app/sql_guard.py	Strips comments and empties string literals on a sanitised copy, then rejects anything that is not a single statement beginning with <code>SELECT</code> or <code>WITH</code> . Forbidden keywords: <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>UPSERT</code> , <code>MERGE</code> , <code>CREATE</code> , <code>DROP</code> , <code>ALTER</code> , <code>TRUNCATE</code> , <code>RENAME</code> , <code>ATTACH</code> , <code>DETACH</code> , <code>PRAGMA</code> , <code>VACUUM</code> , <code>REINDEX</code> , <code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code> , <code>RELEASE</code> , plus the <code>LOAD_EXTENSION()</code> function. <code>REPLACE INTO</code> (a write) is distinguished from <code>REPLACE()</code> (a read-only string function) by whether it is followed by an opening parenthesis.	Fails fast, with a clear, actionable message the model can act on during its one repair attempt.
2. Read-only connection app/data.py	SQLite is opened with the <code>mode=ro</code> URI parameter (via <code>Path.as_uri()</code> , required for correctness on Windows paths containing spaces).	The connection physically cannot write , regardless of what text reaches it — the guarantee of last resort.

Guards against silently wrong (not crashing) SQL

The guard also rejects two classes of query that would execute cleanly and return a plausible but wrong answer — considered the most dangerous failure mode, because it does not look like a failure:

Rejected pattern	Why
<code>date('now')</code> , <code>CURRENT_DATE</code> , <code>CURRENT_TIMESTAMP</code> and relatives	The dataset ends in March 2024; a query anchored to the real clock matches nothing, and "no tickets this week" reads as a plausible true answer.
A month offset applied before <code>'start of month'</code> , e.g. <code>datetime(anchor, '-1 month', 'start of month')</code>	SQLite applies modifiers in order: 30 March minus one month is "30 February," normalised to 1 March — so "last month" silently resolves to the current month. The correct order is <code>'start of month'</code> first.

Answer verification (grounding check)

A prompt instructing the model not to invent figures is a request, not a guarantee. `app/grounding.py` independently verifies every number in a model-written answer — expressed in digits or in words, up to ninety-nine — against the evidence actually shown to the model: the result rows, the anomaly report, the question itself, and the row-count framing. Any number with no traceable source triggers discarding the narration in favour of a deterministic, factually correct summary sentence. The check deliberately allows rounding, derivable percentages (only when written as a percentage), small integers (≤ 10 , treated as ordinary prose), and four-digit years — a conservative bias toward not rejecting good answers.

Additional operational safeguards

- **Query timeout** — a 5-second execution budget per statement (via SQLite's progress handler), protecting against a syntactically valid but pathological query such as an unbounded recursive CTE, which text validation cannot catch because it is not, in fact, invalid SQL.
- **Row cap** — 500 rows maximum per query, enforced both by an injected `LIMIT` clause and independently at fetch time.
- **Fixed refusal wording** — when the model declines to answer (an out-of-scope or destructive request), its own generated prose is never shown verbatim; a single fixed refusal message is returned instead. This closes a defect observed during development, where a declined question was answered with a multi-paragraph essay drawn from the model's general knowledge.
- **CORS restricted** to the configured UI origin only (`localhost` / `127.0.0.1` at the configured UI port) — not opened to all origins.
- **Retry policy** — the SDK's own automatic retry is disabled. Dropped connections and 5xx responses are retried with jittered exponential backoff (base 0.5s, ceiling 4s); rate limits (429) and timeouts are never retried, because retrying either cannot succeed and wastes a shared per-minute token budget.
- **Secrets handling** — the API key is read only from environment configuration, is optional (the system degrades gracefully without it), and is never returned in any response.

Explicit limitation, stated rather than implied: SQL validation by text inspection is inherently approximate. It is deliberately conservative and is **not relied upon as the sole protection** — the read-only database connection is the mechanism that actually makes a write impossible, independent of whether the text guard is ever bypassed.

7. API Surface and Configuration

Endpoints

Method	Path	Purpose	Needs API key
GET	/health	Service status, version, row count, source file, <code>AS_OF</code> , whether the LLM is configured	No
GET	/schema	Columns, types, allowed enum values, available detectors	No
GET	/anomalies	Run the anomaly detectors; optional <code>kind</code> and <code>window_days</code> filters	No
POST	/query	Ask a natural-language question	Yes

Error-to-status-code mapping

Status	Meaning	Caller action
422	Malformed request — empty/over-long question, unknown detector name, non-positive window	Fix the request; rejected before any token is spent
429	Provider rate limit reached (carries a <code>Retry-After</code> header)	Wait, then retry
502	Provider reached but failed, or timed out	Retry
503	No API key configured, or data temporarily unavailable	Configure a key, or retry shortly
500	Unexpected error (including a fault inside a detector)	Report it

Configuration reference

Variable	Default	Notes
GROQ_API_KEY	<i>(none)</i>	Optional. Without it, <code>/query</code> returns 503; everything else works normally.
GROQ_MODEL	openai/gpt-oss-120b	Free-tier alternatives documented in <code>.env.example</code>
LLM_TIMEOUT_SECONDS	30	Per-call wall-clock limit; never retried on expiry
LLM_MAX_RETRIES	2	Retries for transient (connection/5xx) failures only, range 0–5
CSV_PATH	data/support_tickets.csv	Resolved relative to the repository root
AS_OF	<i>(blank → auto)</i>	Blank anchors to the dataset's own latest timestamp; must carry no timezone
IQR_MULTIPLIER	1.5	Tukey-fence multiplier for resolution-time outliers
SLA_BREACH_HOURS	24	Age threshold for an unresolved High/Critical ticket

MAX_RESULT_ROWS	500	Hard cap on rows a generated query may return
QUERY_TIMEOUT_SECONDS	5	Execution budget per SQL statement
API_HOST	127.0.0.1	
API_PORT	8000	
UI_PORT	8501	
LOG_LEVEL	INFO	DEBUG additionally logs the model's tool choice, executed SQL, and each detector's decision

Every setting is validated at process startup by a typed Pydantic model with `extra="forbid"`: an unrecognised key or an out-of-range value fails immediately, with a message naming the field, rather than surfacing mid-request.

8. Evaluation and Performance

Automated test suite

381 tests, run offline with no network access and no API key, using a fake `ChatClient` that returns scripted model responses:

Test file	Tests	Covers
test_llm.py	75	Tool-calling orchestration, retry policy, repair attempts, provider-error translation
test_sql_guard.py	62	SQL validation: forbidden keywords, wall-clock rejection, month-overflow rejection
test_data.py	54	CSV ingestion, NULL coercion, AS_OF anchoring, read-only connections
test_anomalies.py	42	Both anomaly detectors, registry, time windows
test_api.py	37	All four endpoints, status-code mapping, graceful degradation
test_prompts.py	33	System/narration prompt construction and token budgets
test_config.py	26	Settings validation, blank/placeholder normalisation
test_grounding.py	23	Numeric grounding checks, including number words and percentages
test_ui_formatting.py	19	Display/escaping logic used by the Streamlit UI
test_run.py	10	Launcher: port checks, startup ordering
Total	381	Runs in under 10 seconds

Verified by direct execution of the suite against the shipped virtual environment at the time this document was produced: 381 passed, 0 failed.

Live benchmark against the real model

A 50-question benchmark (`docs/BENCHMARK_QUESTIONS.md`) is run against the live Groq API, with expected answers generated from the data and cross-checked against independent SQL — never typed by hand.

Verdict	Count
Passed	41
Failed	0
Needs human review (refusals, open-ended explanations)	9

Automatic pass rate: 100% of the 41 machine-gradable questions. This was reached after two earlier same-day runs scored 95% and 98%; every failure was traced to a specific root cause and closed with a regression test before being counted as fixed — named months read incorrectly, month arithmetic that silently substituted the current month for the previous one, and number-words evading the grounding check.

Evaluation honesty note. The final 100% run predates one grounding-check fix, made afterward, that affected exactly three of its answers (which were correct in figures but rendered as plain deterministic

summaries rather than natural prose). All three questions were re-confirmed live in the UI after the fix, with the model's own phrased answer retained in each case. The benchmark result file itself records the run as it happened and was not edited retroactively.

9. Known Limitations and Risks

Correctness

- **The grounding check verifies numbers, not meaning.** It proves every figure in an answer came from the actual query result; it cannot prove the SQL answered the intended question. A model that misreads a question can produce a valid query against the wrong interpretation, compute a correct-looking but wrong answer, and pass verification. This is the single most important limitation of the design.
- The model has, during development, misread "high-priority" as excluding Critical tickets, and computed "last month" as the current month — both fixed, but structurally similar misreadings of unseen phrasing remain possible.
- LLM output is non-deterministic even at temperature 0; the same question can occasionally be routed or phrased differently across runs.
- The grounding check's permissiveness (rounding, small integers, derivable percentages) is a deliberate trade-off that could in principle let a coincidentally-matching fabricated number through, though none has been observed.
- A pinned `AS_OF` cannot reconstruct a ticket's status as of that moment — the CSV stores only each ticket's final status, so a ticket resolved after `AS_OF` still appears resolved.

Evaluation

- The benchmark's 50 questions were authored by the same person who used them to find and fix bugs, so the system has, in effect, been tuned against its own test set. An independent evaluator's phrasing has not yet been tested against it.
- The unit-test suite exercises a scripted fake model, never the real one — it proves the code handles a given model response correctly, not how the live model will actually respond.
- 9 of 50 benchmark questions require human judgement and are not auto-scored.
- UI rendering and the launcher's process-management behaviour were verified by manual execution rather than automated end-to-end tests, alongside their unit-tested sub-components.

Operational

- **Free-tier rate limits.** Groq's free tier allows 8,000 tokens/minute and 200,000 tokens/day; rapid successive questions can trigger a 429, and only the per-minute limit is visible in response headers — the daily limit is invisible until it is hit.
- **Token counts can be partly estimated** when the provider rejects a request without reporting usage (e.g. a declined tool call); such figures are flagged as estimated rather than presented as exact.
- **Single external dependency.** Natural-language querying stops working if Groq is unreachable or discontinues the configured model; the deterministic endpoints (`/health`, `/schema`, `/anomalies`) are unaffected.

- **Fixed dataset and schema** — there is no ingestion path for a different file's structure without code changes.
- Narration is shown at most 20 rows per result; every row is still returned in the API response, and the written answer always states the true total when it describes only a sample.

Security and scale

- SQL validation is text-based and inherently approximate (stated plainly in the code's own documentation); it is a fail-fast convenience layered on top of the read-only connection, not a substitute for it.
- **Single-user, local-only.** No authentication, no per-user rate limiting; both services bind to `localhost` and are not designed for network exposure.
- SQLite is used in-process; suitable for 500 rows, not built for large datasets or many concurrent users.

10. Versioning and Change History

Stability commitment (Semantic Versioning)

A change that breaks any of the following requires a 2.0.0 release:

- The HTTP API — all four endpoints, their parameters, every documented request/response field, and the meaning of each status code (422, 429, 502, 503, 500).
- Configuration — the names and meanings of the fourteen settings in `.env.example`.
- Startup — `python run.py` launches the API and UI together.

Deliberately **not** covered, and free to change in a minor release: the exact wording of model-written answers, log messages, UI layout, and error-message text (whose status codes remain covered).

Release history

Version	Date	Summary
0.1.0	2026-09-16	Initial scaffolding; model selection confirmed against the live Groq API rather than assumed from documentation.
0.2.0	2026-09-17	Deterministic core: ingestion, SQL guard, both anomaly detectors — runs with no LLM, no network, no API key.
0.3.0	2026-09-17	Natural-language layer: bounded two-step tool-calling pipeline; all five brief sample questions answer correctly.
0.4.0	2026-09-17	HTTP layer: four endpoints, typed contract, distinct status code per failure mode.
0.5.0	2026-09-17	Streamlit UI as a thin HTTP client of the API, with evidence shown alongside every answer.
0.6.0	2026-09-17	Single-command startup (<code>python run.py</code>) with health-gated launch order and clean shutdown.
0.7.0	2026-09-17	Grounding verification introduced; 50-question live benchmark and explicit retry policy added.
0.8.0	2026-09-18	Full code review; every benchmark failure traced to root cause and fixed; redesigned UI; 100% automatic benchmark pass rate reached.
1.0.0	2026-09-18	First stable release. No functional change from 0.8.0 — declared stable on an already-verified build rather than on a final round of untested edits.

11. Dependencies

All versions pinned exactly, so an evaluator's environment resolves to the same package set the system was built and tested against.

Package	Version	Role
fastapi	0.141.1	REST framework; generates the <code>/docs</code> OpenAPI UI
uvicorn	0.53.0	ASGI server
pydantic	2.13.5	Typed request/response and settings models
pydantic-settings	2.15.0	Environment/.env-backed settings loading
python-dotenv	1.2.3	Reads <code>.env</code> in <code>run.py</code> before subprocesses spawn
groq	1.7.0	Official SDK; OpenAI-compatible tool calling
pandas	3.0.5	Anomaly-detection statistics (vectorised quantiles)
streamlit	1.64.0	Web UI
altair	6.3.0	Severity charts, declared explicitly rather than relied on transitively
httpx	0.28.1	UI → API calls; also used by FastAPI's test client
pytest	9.1.1	Test runner

SQLite access uses Python's standard library `sqlite3` module — no separate database server or dependency. No Docker or containerisation is required or used.

12. Ownership and Maintenance

Field	Value
Author / maintainer	Vijaya Karthik
Built for	DOTMappers IT Pvt. Ltd. — AI Engineer internship assessment
Build window	2026-09-16 18:30 to 2026-09-18 18:30 (48 hours)
Current status	Complete, submitted, stable release 1.0.0
Repository documentation	<code>README.md</code> (architecture, setup, API reference), <code>CHANGELOG.md</code> (full version-by-version history), <code>docs/PROJECT_DOCUMENTATION.md</code> (build retrospective), <code>docs/BENCHMARK_QUESTIONS.md</code> / <code>BENCHMARK_RESULTS.md</code> (evaluation detail)
License / distribution	Submitted as a private assessment deliverable; not published for general use

This System Card reflects the system as released at **version 1.0.0**. Every quantitative claim in this document — the test count, the file-by-file test breakdown, the benchmark result, endpoint behaviour, configuration defaults, and safety-guard logic — was verified directly against the project's source code and its executed test suite at the time of writing, rather than transcribed from prior documentation alone.